# DOCUMENTAÇÃO TÉCNICA DO PROJETO INTEGRADOR DE ENGENHARIA DE SOFTWARE

## Plataforma Ecosol

### Rede de Economia Solidária de Niterói

| | |
|---|---|
| **Equipe** | Não se aplica (equipe sem nome definido) |
| **Integrantes** | Jean Macedo<br>Vinícius Pontes<br>Pietro Gomes |
| **Tema/Contexto da solução** | Economia Solidária: plataforma web de divulgação e cadastro dos coletivos da rede de Niterói, em parceria com a Casa Paul Singer (Centro Público de Economia Solidária) e a ITES/IFRJ |
| **Repositório Git** | Organização: https://github.com/Plataforma-Ecosol<br>Backend: https://github.com/Plataforma-Ecosol/ecosol-backend<br>Frontend: https://github.com/Plataforma-Ecosol/ecosol-frontend<br>Infraestrutura: https://github.com/Plataforma-Ecosol/ecosol-infra |
| **Wiki de planejamento e documentação do projeto** | [URL da Wiki do projeto no GitHub] |
| **Quadro Kanban** | https://github.com/orgs/Plataforma-Ecosol/projects/1<br>(GitHub Project “Plataforma ES Niterói (SCRUM)”) |
| **Semestre** | 2026.2 |

## CONTROLE DE VERSÕES DA DOCUMENTAÇÃO

| Versão | Data | Atividade/entrega | Responsável(is) | Principais alterações |
|---|---|---|---|---|
| 1.0 | 09/11/2026 | AV1: documentação técnica inicial | Jean Macedo, Vinícius Pontes e Pietro Gomes | Primeira versão: identificação, contexto e escopo, requisitos (RF, RNF e RN), diagramas C4 (níveis 1 e 2), diagrama de classes, modelo lógico, estilo arquitetural e decisões tecnológicas, com base no MVP já implementado. |

# 1. CONTEXTO E OBJETIVO DA SOLUÇÃO

## 1.1 Contexto do problema

A Economia Solidária (ES) reúne coletivos e empreendimentos que trabalham de forma autogestionária, como grupos de artesanato, alimentação, agroecologia e serviços. Em Niterói, essa rede é acompanhada pelo Centro Público de Referência em Economia Solidária, a Casa Paul Singer (também chamada de Casa Azul), com apoio da Incubadora Tecnológica de Empreendimentos Solidários (ITES) do IFRJ Campus Niterói. A plataforma é desenvolvida na ITES com bolsa PIBITI/CNPq, como uma Tecnologia Social de código aberto.

Hoje o cadastro dos coletivos fica em uma planilha que mistura, na mesma linha, dados das pessoas e dados do coletivo. Isso dificulta a consulta, gera retrabalho e deixa dados pessoais e sensíveis (como cor/raça, renda e identidade de gênero) sem proteção adequada. Também não existe um canal público em que consumidores, parceiros e os próprios coletivos descubram quem faz parte da rede, o que cada um produz e como entrar em contato. A proposta é fortalecer a rede pela descoberta: a plataforma mostra quem existe e como falar com cada coletivo, e a negociação acontece fora dela.

Públicos interessados e potenciais usuários:

- **Equipe da Casa Paul Singer:** administra o cadastro da rede (coletivos, pessoas, categorias, eventos e pontos de interesse) e decide o que pode ser exibido ao público.
- **Coletivos e empreendimentos solidários:** ganham um perfil público, com fotos e os contatos que autorizaram divulgar.
- **Consumidores e interessados:** buscam coletivos por categoria ou bairro, consultam a agenda de feiras, encontros e formações e o mapa de pontos da ES, inclusive pelo celular.
- **ITES/IFRJ:** incubadora responsável pelo desenvolvimento e pela continuidade da solução.

Principais informações registradas ou consultadas: dados dos coletivos (identificação pública, contatos com autorização de exibição, dados cadastrais e endereço de uso interno), dados das pessoas que compõem cada coletivo (uso exclusivo da equipe), categorias, eventos da rede e pontos de interesse da ES, como órgãos públicos, lojas físicas e feiras do Circuito Araribóia.

Cenários de uso típicos: a equipe cadastra um coletivo depois da entrevista e marca quais contatos podem aparecer; uma pessoa procura, no celular, coletivos de alimentação no seu bairro; alguém compartilha o link do perfil de um coletivo no WhatsApp; um visitante consulta a agenda para achar a próxima feira e vê no mapa onde ela acontece.

Fontes utilizadas: a planilha de cadastro usada pela Casa Paul Singer, que deu origem ao modelo de dados; a reunião de alinhamento com a equipe da Casa Azul (setembro de 2026), que definiu a identidade visual e as prioridades seguintes; as Sprint Reviews com a ITES e a Casa Paul Singer; e o PRD Técnico v4.1 do projeto, mantido no repositório ecosol-backend.

| | |
|---|---|
| O problema de | o cadastro dos coletivos da Economia Solidária de Niterói ficar em uma planilha que mistura dados de pessoas e de coletivos, sem um canal público para divulgar a rede |
| afeta | os coletivos e empreendimentos solidários, a equipe da Casa Paul Singer e os consumidores e interessados na Economia Solidária |
| cujo impacto é | a baixa visibilidade dos coletivos e a dificuldade de serem encontrados por consumidores e uns pelos outros, o retrabalho e o risco de erro no cadastro, e a exposição indevida de dados pessoais e sensíveis |
| uma boa solução seria | uma plataforma web aberta, com um portal público de consulta (coletivos, agenda e mapa) e uma área administrativa com login para a equipe, que proteja os dados pessoais conforme a LGPD |

### 1.2 Escopo do projeto (proposta de solução tecnológica)

A Plataforma Ecosol é uma aplicação web que dá visibilidade aos coletivos da Economia Solidária de Niterói e substitui a planilha por um cadastro estruturado. O público consulta as informações sem precisar de login. A equipe da Casa Paul Singer mantém os dados em uma área administrativa protegida e decide, campo a campo, o que pode ser exibido. A versão inicial (MVP) foi concluída em setembro de 2026 e está publicada em ambiente de homologação, com dados fictícios: site em https://ecosol-frontend.vercel.app e API em https://ecosol-backend-homolog.onrender.com.

Módulos do MVP:

- **Portal público (sem login):** página inicial com os números da rede, coletivos em destaque e próximos eventos; listagem de coletivos com busca, filtros por categoria e bairro e paginação; perfil de cada coletivo; agenda de eventos com página de detalhe e galeria; mapa dos pontos de interesse; página “Saiba mais” com a apresentação e os contatos da Casa Paul Singer.
- **Área administrativa (com login):** cadastro, edição e exclusão de coletivos, pessoas, categorias, eventos, pontos de interesse e usuários da equipe, com a identidade visual da Casa Azul. Inclui as chaves de visibilidade pública (coletivo ativo e autorização de cada contato).
- **API REST pública:** somente leitura, com paginação, busca, filtros e ordenação. É a única fonte de dados do portal e pode alimentar um aplicativo no futuro.
- **Proteção de dados (transversal):** nenhum dado de pessoa é público; o endereço do coletivo nunca aparece (só o bairro); contatos só aparecem com autorização; testes automáticos impedem que um dado sensível vaze pela API ou pela listagem administrativa.

Fora do MVP: qualquer transação (escambo, moeda social Araribóia ou pagamentos); autocadastro, login público e área do coletivo; registro de parcerias e troca de mensagens entre coletivos; e exibição da localização exata dos coletivos. Esses itens ficam registrados no quadro como “Fora do escopo” e só voltam ao backlog se forem aprovados em um Spike.

Evolução prevista após o MVP (Sprints 4 a 6, a partir dos pedidos da Casa Azul): e-mail de confirmação de cadastro; etapas de formação em ES por pessoa e cálculo da aptidão do coletivo; e-mail quando o coletivo ficar apto; página pública de privacidade; e publicação em produção. Lista de presença automática, gráficos socioeconômicos e permissões por papel dependem de Spikes antes de virar requisito.

### 1.3 Requisitos da Solução Tecnológica

Os requisitos abaixo descrevem o MVP já implementado. Há dois perfis: o Público Geral, que só consulta e não faz login, e o Administrador, que é a equipe da Casa Paul Singer com acesso à área administrativa.

#### 1.3.1 Requisitos Funcionais (RF)

| Código | Descrição do Requisito Funcional |
|---|---|
| **RF01** | O software deve permitir que qualquer visitante, sem login, veja a página inicial com a apresentação da rede, os totais de coletivos, pontos no mapa e próximos eventos, alguns coletivos em destaque e os próximos eventos. |
| **RF02** | O software deve permitir que o visitante liste os coletivos ativos da rede, em páginas de 20 itens. |
| **RF03** | O software deve permitir que o visitante busque coletivos por texto no nome e na descrição. |
| **RF04** | O software deve permitir que o visitante filtre os coletivos por uma ou mais categorias e por bairro, combinando os filtros com a busca, e mantenha busca, filtros e página no endereço (URL), para que a listagem possa ser compartilhada. |
| **RF05** | O software deve permitir que o visitante veja o perfil público de um coletivo pelo endereço com o seu identificador textual (slug), com nome, foto, categorias, descrição, bairro, site e os contatos cuja exibição foi autorizada. |
| **RF06** | O software deve redirecionar o endereço antigo de um coletivo para o novo quando a equipe alterar o slug. |
| **RF07** | O software deve permitir que o visitante consulte a agenda de eventos da ES, separada em “próximos” e “já aconteceram”, com busca e filtros por período e por bairro. |
| **RF08** | O software deve permitir que o visitante veja a página de um evento, com data e horário, local, bairro, descrição, link e galeria de imagens. |
| **RF09** | O software deve exibir em um mapa interativo os pontos de interesse da ES (órgãos da ES, lojas físicas e feiras do Circuito Araribóia), com os dados do ponto ao clicar nele, e uma lista dos mesmos pontos. |
| **RF10** | O software deve exibir a página “Saiba mais”, com a apresentação e os contatos da Casa Paul Singer. |
| **RF11** | O software deve permitir que membros da equipe entrem na área administrativa com usuário e senha e encerrem a sessão. |
| **RF12** | O software deve permitir que a equipe cadastre, consulte, edite e exclua coletivos, com foto, categorias, contatos, dados cadastrais, endereço e observações. |
| **RF13** | O software deve permitir que a equipe cadastre, consulte, edite e exclua pessoas, sempre vinculadas a um coletivo, incluindo o bloco socioeconômico e de identidade, de uso interno. |
| **RF14** | O software deve permitir que a equipe cadastre, edite e exclua categorias, mostrando quantos coletivos há em cada uma. |
| **RF15** | O software deve permitir que a equipe cadastre, edite e exclua eventos, com uma galeria de imagens ordenada. |
| **RF16** | O software deve permitir que a equipe cadastre, edite e exclua pontos de interesse, com tipo, latitude e longitude, endereço, imagem de capa e vínculo opcional com um coletivo. |
| **RF17** | O software deve permitir que a equipe controle o que é público: ativar ou desativar coletivos, eventos e pontos, e autorizar, contato a contato, a exibição de telefone, e-mail e Instagram de cada coletivo. |
| **RF18** | O software deve permitir que a equipe gerencie as contas de usuário da área administrativa e as suas permissões. |

| Código | Descrição do Requisito Funcional |
|---|---|
| **RF19** | O software deve disponibilizar uma API REST pública, somente leitura, de coletivos, eventos e pontos de interesse, com paginação, busca, filtros e ordenação, além das opções dos painéis de filtro (categorias e bairros em uso). |

### 1.3.2 Requisitos não funcionais (RNF)

| Código | Descrição do Requisito Não Funcional |
|---|---|
| **Requisitos de Comunicação de Dados, Interface e Interoperabilidade:** | |
| **RNF01** | Frontend e backend devem se comunicar apenas por HTTP, por meio de uma API REST com dados em JSON, chaves em snake_case e datas no padrão ISO 8601 (data e hora sempre com fuso horário). |
| **RNF02** | As listagens da API devem usar o envelope de paginação (count, next, previous, results), com 20 itens por página por padrão e no máximo 100. |
| **RNF03** | Todo acesso do portal à API deve acontecer no servidor do Next.js; nenhum componente executado no navegador deve chamar a API do Django. |
| **Requisitos de Confiabilidade:** | |
| **RNF04** | Toda listagem deve ter ordenação estável, usando a chave primária como último critério de desempate, para que nenhum registro se repita ou desapareça entre páginas. |
| **RNF05** | O endereço de um coletivo já divulgado não pode deixar de funcionar após a troca do slug: o endereço antigo deve redirecionar de forma permanente (API 301, site 308). |
| **Requisitos de Desempenho e Robustez:** | |
| **RNF06** | As listagens da API devem responder em menos de 500 ms, apoiadas por paginação, índices e consultas sem o problema de N+1. |
| **RNF07** | O portal deve manter em cache as respostas da API por 60 segundos nas listagens e 300 segundos nas páginas de detalhe. |
| **Requisitos de Disponibilidade:** | |
| **RNF08** | Cada merge na branch staging deve publicar automaticamente o ambiente de homologação, que deve usar banco e armazenamento próprios, separados dos de produção, e apenas dados fictícios. |
| **Requisitos de Manutenibilidade:** | |
| **RNF09** | Todo pull request deve passar pela integração contínua (backend: ruff, paridade entre models e migrations e pytest; frontend: ESLint, checagem de tipos, Vitest e next build). PR com falha não pode ser mesclado em staging nem em main. |
| **RNF10** | A branch main deve ser protegida: código entra só por pull request revisado e aprovado por outra pessoa, com commits no padrão Conventional Commits. |
| **RNF11** | O código e a documentação devem estar em português, para que a solução seja compreendida e mantida por quem herdar o projeto. |
| **Requisitos de Portabilidade:** | |
| **RNF12** | O sistema deve subir completo (banco, API e portal) com um único comando via Docker Compose, sem instalação manual. |
| **RNF13** | Nenhum componente essencial deve prender o projeto a um fornecedor: o Supabase é usado só como PostgreSQL e armazenamento, e pode ser trocado por outro PostgreSQL apenas mudando a configuração. |
| **RNF14** | O portal deve funcionar nas versões atuais dos navegadores Chrome, Firefox, Safari e Edge, em computador e celular. |
| **Requisitos de Segurança:** | |

| Código | Descrição do Requisito Não Funcional |
|---|---|
| **RNF15** | Nenhuma rota de escrita pode ser acessada sem sessão autenticada da equipe; a API pública aceita apenas leitura (GET). |
| **RNF16** | A autenticação deve usar a sessão do Django com cookie httpOnly e, em homologação e produção, cookies apenas por HTTPS; as senhas devem ser guardadas com hash. |
| **RNF17** | Os serializers públicos devem declarar os campos de forma explícita. Dados de pessoas, endereço completo e dados cadastrais do coletivo nunca podem sair na API pública, o que deve ser garantido por teste automatizado de regressão no CI. |
| **Requisitos de Usabilidade:** | |
| **RNF18** | O portal deve ser responsivo (mobile first) e plenamente utilizável em smartphones básicos. |
| **RNF19** | A interface deve seguir a identidade visual da Casa Azul: azul (#33469B), dourado (#BD9D55) e bege (#F6F0DC), com a logo da Casa Azul, no portal e na área administrativa. |
| **RNF20** | As páginas públicas devem ser indexáveis por buscadores e gerar prévia ao serem compartilhadas, com renderização no servidor, endereços por slug, metadados, sitemap e robots. O ambiente de homologação não deve ser indexado. |
| **Restrições de Projeto e Tecnológicas:** | |
| **RNF21** | O backend deve ser desenvolvido em Python com Django e Django REST Framework, o portal em Next.js com TypeScript, e o banco de dados deve ser PostgreSQL. |
| **RNF22** | O código deve ser aberto, sob a licença GNU GPLv3. |
| **Restrições Legais:** | |
| **RNF23** | O tratamento de dados deve seguir a LGPD (Lei nº 13.709/2018): dados sensíveis ficam apenas na área administrativa e não podem virar coluna, filtro ou campo de busca na listagem; a base legal, a política de retenção e o atendimento aos titulares devem estar documentados antes do primeiro cadastro com dado real. |
| **RNF24** | A interface deve observar os critérios de acessibilidade aplicáveis (WCAG), com texto alternativo nas imagens, rótulos para leitor de tela e navegação por teclado. |

### 1.3.3 Regras de negócio (RN)

| Código | Descrição da regra de negócio |
|---|---|
| **RN01** | Um coletivo válido tem no mínimo 3 pessoas e é interfamiliar (pelo menos duas famílias). A verificação é feita pela equipe antes do cadastro, e não pelo sistema: o que a equipe cadastra é considerado válido. (RF12) |
| **RN02** | Somente coletivos, eventos e pontos marcados como ativos aparecem para o público. Um coletivo inativo some da listagem, e o seu perfil, inclusive por um slug antigo, responde como inexistente (404). (RF02, RF05, RF17) |
| **RN03** | Telefone, e-mail e Instagram do coletivo só são exibidos quando a equipe autoriza cada um deles. Sem autorização, o dado é omitido da resposta, e não enviado como vazio. (RF05, RF17) |
| **RN04** | Nenhum dado de pessoa é exibido no portal ou na API pública, em nenhuma hipótese. (RF13) |
| **RN05** | O bairro do coletivo é público, mas o endereço completo nunca é. Apenas pontos de interesse têm latitude e longitude e aparecem no mapa. (RF05, RF09) |
| **RN06** | Toda pessoa pertence a exatamente um coletivo. Um coletivo que ainda tem pessoas vinculadas não pode ser excluído. (RF12, RF13) |
| **RN07** | O slug é único. Quando a equipe muda o slug de um coletivo, o anterior fica registrado e passa a redirecionar para o novo. Um slug nunca aponta para dois coletivos. (RF06) |

| Código | Descrição da regra de negócio |
|---|---|
| **RN08** | A situação do coletivo (regular, em transição ou encerrado) é só um rótulo cadastral e não interfere na visibilidade, que é decidida pelo campo “ativo”. (RF12, RF17) |
| **RN09** | Não há autocadastro. Coletivos e pessoas são cadastrados apenas pela equipe, e as contas de acesso são criadas pela própria equipe. (RF11, RF18) |
| **RN10** | A data de fim de um evento, quando informada, não pode ser anterior à data de início. Um evento continua entre os “próximos” enquanto ainda não terminou. (RF07, RF15) |
| **RN11** | A foto do coletivo tem no máximo 5 MB e só é publicada com a autorização de uso de imagem das pessoas que aparecem nela. (RF12) |
| **RN12** | Os campos socioeconômicos e de identidade das pessoas não podem ser usados como coluna, filtro ou busca na listagem da área administrativa. (RF13) |
| **RN13** | O tipo de um ponto de interesse é um entre: órgão da ES, loja física ou feira do Circuito Araribóia. A latitude fica entre -90 e 90, e a longitude, entre -180 e 180. (RF09, RF16) |
| **RN14** | Os filtros públicos de categoria e bairro só oferecem valores em uso por coletivos ativos, para não revelar a existência de coletivos fora do ar. (RF04, RF19) |

## 2. DESIGN E ARQUITETURA DA SOLUÇÃO

### 2.1 Modelo de arquitetura — contexto do sistema (C4, nível 1)

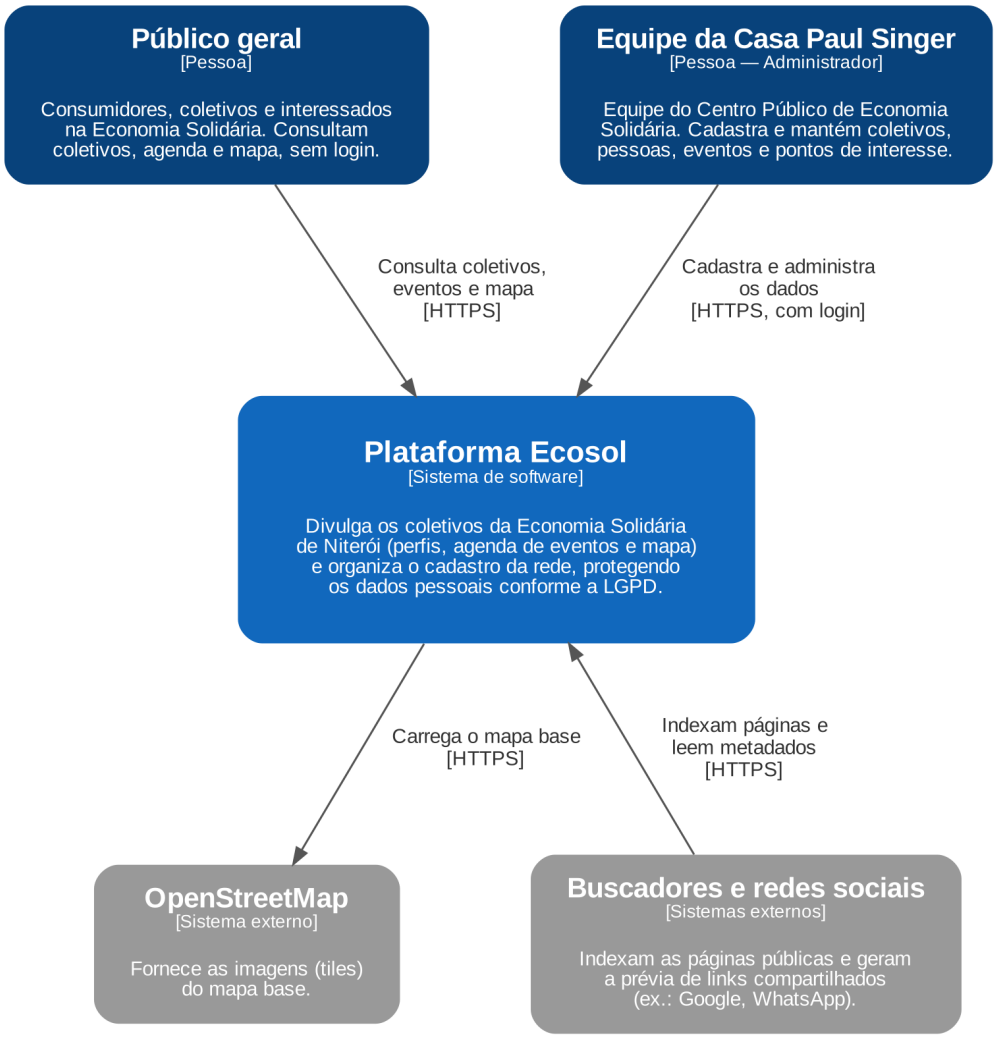



*Figura 1 – Diagrama de contexto da Plataforma Ecosol (C4, nível 1).*

Dois perfis usam o sistema: o Público Geral, que consulta coletivos, agenda e mapa sem login, e a equipe da Casa Paul Singer, que administra os dados com login. Os sistemas externos são o OpenStreetMap, que fornece o mapa base, e os buscadores e redes sociais, que indexam as páginas e montam a prévia dos links. Banco de dados e armazenamento de imagens fazem parte da solução e aparecem no nível 2.

## 2.2 Modelo de arquitetura — contêineres (C4, nível 2)

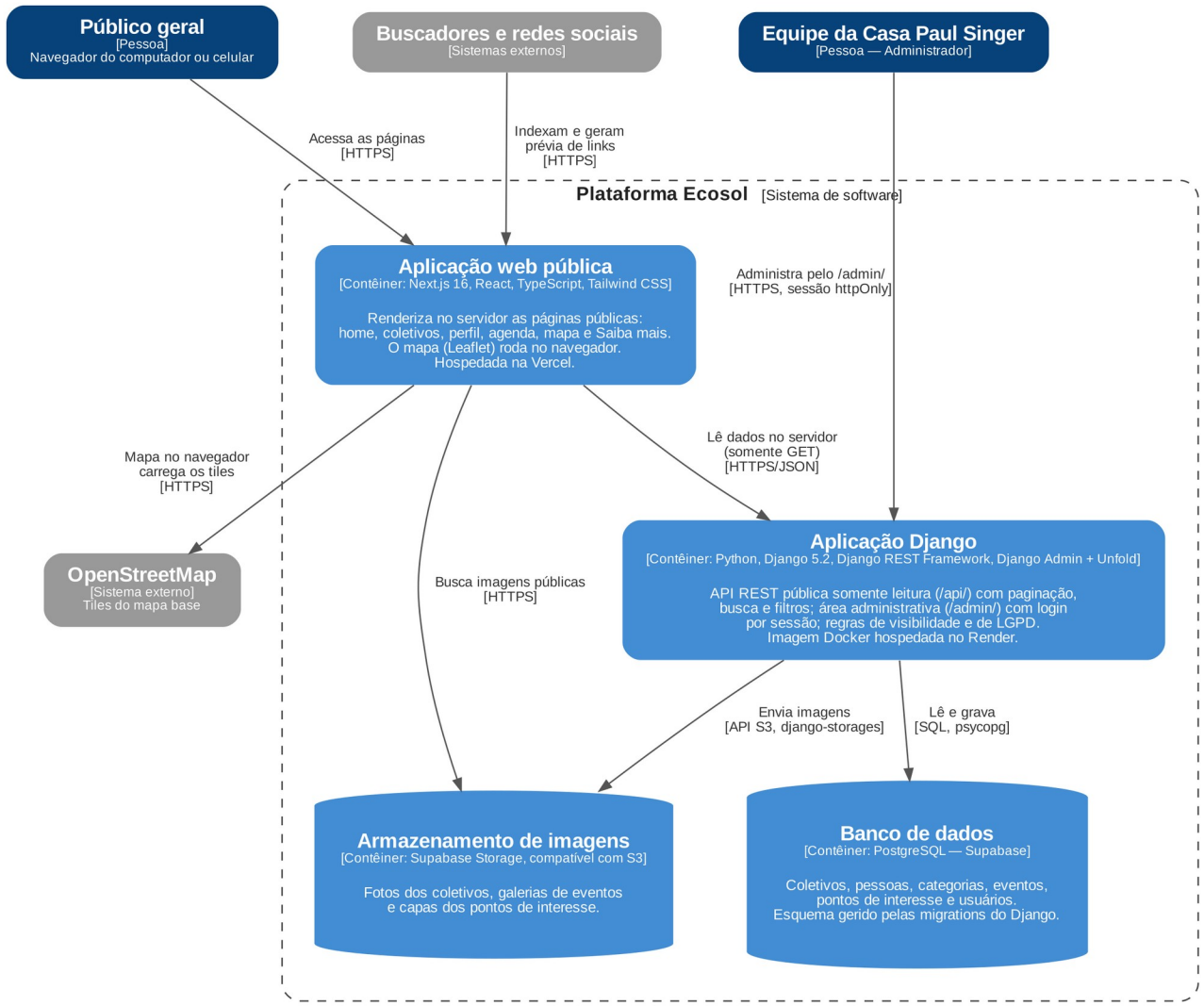



*Figura 2 – Diagrama de contêineres da Plataforma Ecosol (C4, nível 2).*

A solução tem quatro contêineres. A aplicação web pública (Next.js, na Vercel) renderiza as páginas no servidor e lê os dados da aplicação Django só por GET. A aplicação Django (no Render, em Docker) concentra a API REST pública e a área administrativa, e é a única que grava no banco PostgreSQL e no armazenamento de imagens, ambos no Supabase. As imagens públicas são lidas direto do armazenamento. No ambiente local, o repositório ecosol-infra sobe banco, API e portal com um comando.

## 2.3 Diagrama de classes do domínio da solução

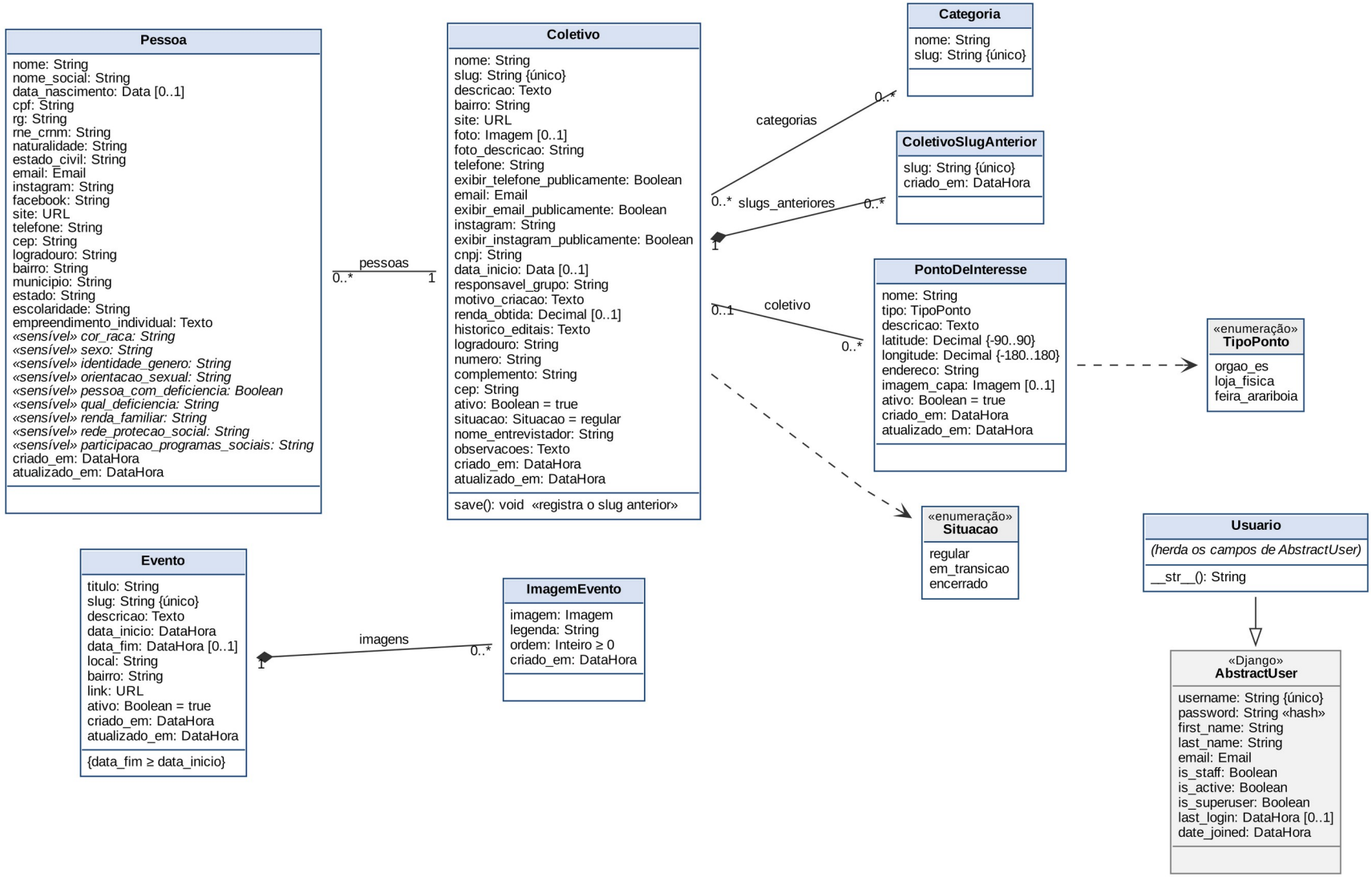



*Figura 3 – Diagrama de classes do domínio (app "rede" do backend). Versão vetorial em docs/diagramas/diagrama-classes.svg.*

## 2.4 Modelo lógico (relacional) do banco de dados

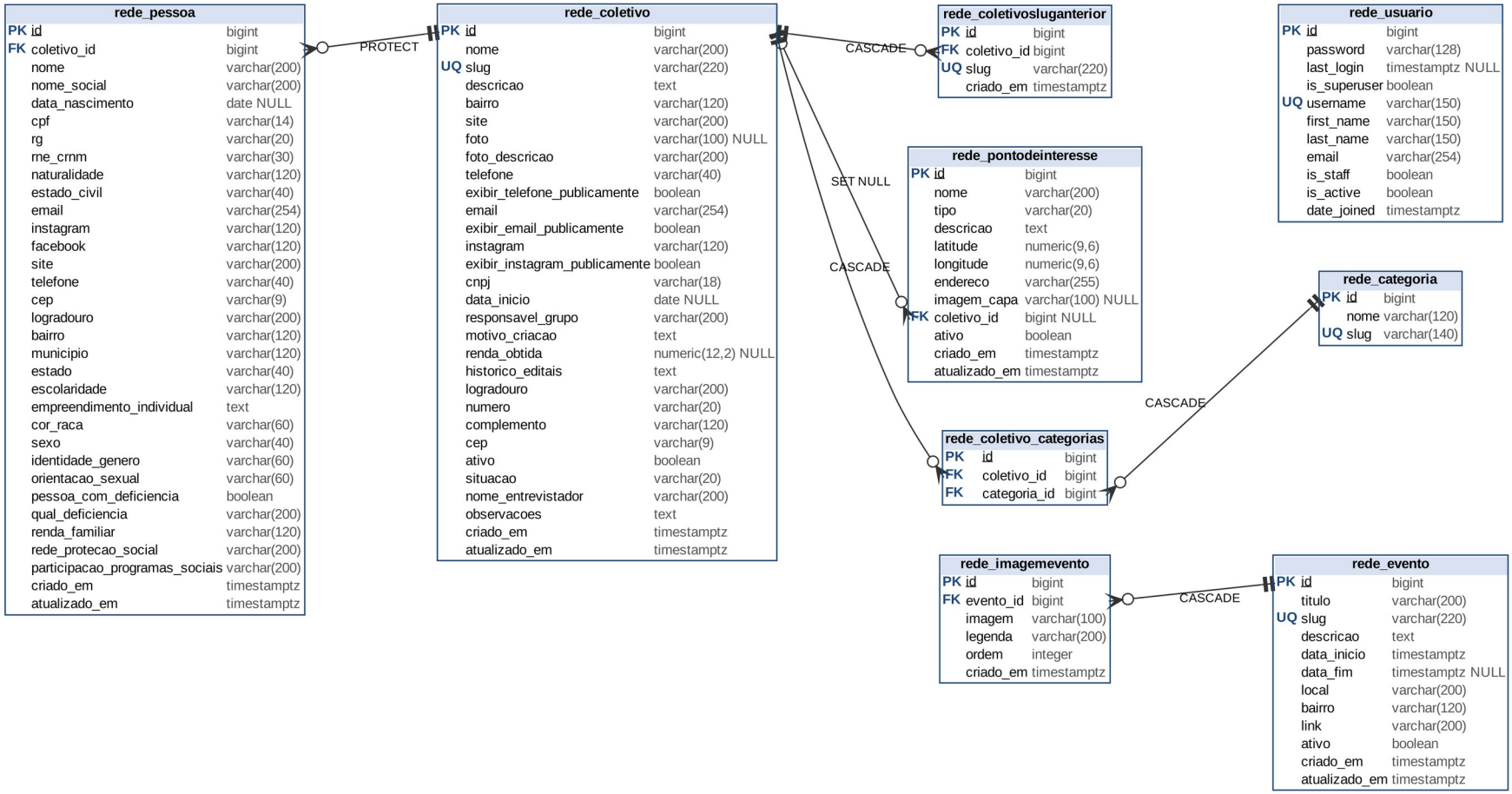



*Figura 4 – Modelo lógico relacional (PostgreSQL). PK: chave primária; FK: chave estrangeira; UQ: valor único; colunas sem "NULL" são obrigatórias.*

**Leitura do diagrama de classes (Figura 3):** O Coletivo é o centro do domínio. Uma Pessoa pertence sempre a um Coletivo (associação 1 para 0..*), e um Coletivo pode ter várias Categorias, e cada Categoria vários Coletivos (N:N). O ColetivoSlugAnterior guarda os endereços antigos do coletivo e só existe junto dele (composição), assim como as imagens de um Evento. O PontoDeInteresse é independente e pode, opcionalmente, apontar para um Coletivo que tenha ponto físico de venda. O Usuario representa a equipe administrativa e herda do usuário padrão do Django. Os atributos marcados como «sensível» ficam apenas na área administrativa (RN04 e RN12).

**Leitura do modelo lógico (Figura 4):** As tabelas são criadas pelas migrations do Django, que são a única forma de alterar o esquema (ninguém edita tabela pelo painel do Supabase). Toda tabela tem chave primária substituta (id bigint autoincremental). A relação N:N entre coletivos e categorias usa a tabela associativa rede_coletivo_categorias. As regras de exclusão das chaves estrangeiras seguem as regras de negócio: PROTECT em rede_pessoa (RN06), SET NULL no vínculo opcional do ponto de interesse e CASCADE nos slugs anteriores, nas imagens de evento e na tabela associativa. O banco também aplica a restrição evento_data_fim_apos_inicio (RN10) e índices em rede_coletivo.bairro e rede_coletivo.ativo, usados nos filtros públicos. As tabelas internas do Django (grupos, permissões, sessões e log do Admin) foram omitidas do diagrama.

### 2.5 Estilo arquitetural e justificativa

**Estilo adotado:** arquitetura cliente-servidor desacoplada, com o backend organizado em camadas segundo o padrão MVT (Model-View-Template) do Django e o frontend também organizado em camadas.

No backend, a camada de modelo (models do app “rede”) define as entidades, as validações e as regras ligadas aos dados, como o histórico de slug. A camada de apresentação da API usa serializers e views do Django REST Framework: os serializers definem exatamente quais campos saem em JSON, e as views aplicam permissões, paginação, busca, filtros e ordenação. Na área administrativa, o Django Admin cumpre o papel de View e Template do MVT. A persistência fica no PostgreSQL, acessado só pelo ORM.

No frontend, as rotas do App Router do Next.js formam a camada de páginas; os componentes de interface ficam em uma pasta própria; e toda conversa com a API passa por uma camada de acesso única (lib/api.ts), executada no servidor e tipada com o contrato da API. Assim, a interface nunca depende de detalhes do backend.

**Justificativa:** a descoberta dos coletivos é o objetivo principal, então as páginas públicas precisam ser renderizadas no servidor para serem indexadas e gerarem prévia de link (RNF20), o que levou a um portal separado do backend. A separação também permite que a equipe trabalhe em paralelo em repositórios diferentes e que a mesma API sirva a um aplicativo no futuro. Do lado do servidor, o MVT do Django entrega a área administrativa praticamente pronta, com autenticação e permissões, o que protegeu o prazo do MVP com uma equipe pequena. As camadas ajudam a cumprir a LGPD: como os campos públicos são declarados um a um no serializer, um dado sensível não vaza por acidente (RNF17). A complexidade é controlada: um único backend (monólito modular), sem microsserviços, adequado ao volume de dados da rede e à infraestrutura gratuita.

**Estilo × tecnologias:** o estilo é a forma de organizar a solução (cliente-servidor, camadas e MVT) e não depende das ferramentas. Django, Next.js e PostgreSQL são as tecnologias escolhidas para implementar esse estilo e poderiam ser trocadas por outras equivalentes sem mudar a arquitetura.

## 3. DECISÕES TECNOLÓGICAS

A tabela resume as tecnologias usadas no MVP, a finalidade de cada uma e o motivo da escolha. Os critérios foram: adequação aos requisitos (principalmente indexação, proteção de dados e responsividade), conhecimento da equipe, custo zero ou baixo, código aberto e facilidade de integração entre os componentes.

| Tecnologia | Finalidade | Justificativa |
|---|---|---|
| **Python 3.12 e Django 5.2** | Linguagem e framework do backend | Linguagem que a equipe já domina. O Django traz ORM, migrations, autenticação, permissões e o Admin prontos, o que reduz código e tempo do MVP. |
| **Django REST Framework e django-filter** | API REST pública | Padroniza serializers, paginação, busca, filtros e permissões. Serializers explícitos são a base da proteção de dados na API (RNF17). |
| **Django Admin com Unfold** | Área administrativa | CRUD completo das seis entidades sem escrever telas do zero, com login e permissões nativos. O Unfold aplica a identidade da Casa Azul (RNF19). |
| **PostgreSQL (Supabase)** | Banco de dados relacional | Banco relacional robusto e livre, adequado às relações e restrições do domínio. O Supabase oferece PostgreSQL gerenciado com plano gratuito e é usado só como banco (RNF13). |
| **Supabase Storage, django-storages e boto3** | Armazenamento de imagens | Armazenamento compatível com S3 para fotos de coletivos, eventos e pontos, integrado ao Django sem código próprio. Pode ser trocado por qualquer serviço S3. |
| **Next.js 16, React 19 e TypeScript** | Portal público | Renderização no servidor para indexação e prévia de links (RNF20). O TypeScript reduz erros no contrato entre telas e API. |
| **Tailwind CSS 4, Radix UI e Lucide** | Estilo e componentes da interface | Estilo utilitário e mobile first, fácil de alinhar às cores da Casa Azul. Componentes acessíveis do Radix e ícones do Lucide. |
| **Leaflet, react-leaflet e OpenStreetMap** | Mapa dos pontos de interesse | Bibliotecas livres e sem custo, sem depender de fornecedor, de acordo com a proposta de Tecnologia Social. |
| **Sessão do Django (cookie httpOnly)** | Autenticação da equipe | Só a equipe faz login e o público não escreve. A sessão httpOnly é mais simples e segura que JWT para esse caso. JWT fica como evolução se houver aplicativo. |
| **Docker e Docker Compose** | Empacotamento e ambiente local | Mesmo ambiente para toda a equipe e reaplicação por outras redes com um comando (RNF12). Facilita a migração futura para um servidor próprio (VPS). |
| **Gunicorn e WhiteNoise** | Execução do Django em produção | Servidor de aplicação padrão para Django e entrega dos arquivos estáticos do Admin sem servidor extra. |
| **Vercel e Render** | Hospedagem (homologação) | Planos gratuitos com publicação automática a cada merge na staging (RNF08). A troca do Render por uma VPS e o uso de domínio próprio estão em avaliação. |
| **GitHub, GitHub Actions e GitHub Projects** | Versionamento, CI e gestão Scrum | Organização com repositórios separados, CI bloqueante em todo PR (RNF09), proteção da main e quadro Scrum com sprints, pontos e automações ligadas às issues. |
| **pytest, pytest-django e ruff** | Testes e qualidade do backend | Suíte com testes de models, API, Admin e regressão de LGPD, além de lint, rodando no CI. |
| **Vitest, Testing Library e ESLint** | Testes e qualidade do frontend | Testes de componentes e da camada de acesso à API, lint e checagem de tipos no CI. |